Azure Databricks Architecture — Cheat Sheet

Microsoft Learn module: Understand Azure Databricks architecture

---

1. Account Hierarchy

Azure Databricks organizes resources top-down: **Account** → **Metastore** → **Workspace(s)** → **Catalog** → **Schema** → **Object** (table, view, volume, model, function).

Account

- Top-level construct for managing Azure Databricks across the whole org.
- Manages identity/access, creates & configures workspaces, attaches Unity Catalog metastores, oversees billing and policies.

Metastore (Unity Catalog)

- Top-level container for **metadata**: tables, views, volumes, models, functions, plus the permissions governing them.
- **Region-specific.** Workspaces attached to the same metastore share one unified view of data → centralized governance.
- Unity Catalog uses a **3-level namespace**: `<catalog>.<schema>.<object>`.
- Unlike the legacy Hive metastore (per-workspace), Unity Catalog metastores operate at the **account level** — define access policy once, apply everywhere.

Legacy Hive Metastore

- Old workspace-level metadata store, used before Unity Catalog.
- Once a workspace is UC-enabled, Hive metastore becomes accessible as a catalog named `hive_metastore` → query as `hive_metastore.<schema>.<table>`.
- Databricks recommends **migrating to Unity Catalog** for better security, centralized governance, and built-in auditing.

Workspace

- The collaboration environment where teams run compute workloads: ingestion, interactive exploration, scheduled jobs, ML training.
- You can create **multiple workspaces** per account. Each is isolated per project but shares account-level governance.

2. Control Plane vs. Compute Plane

Azure Databricks splits responsibilities into two planes so orchestration is centralized while data processing stays isolated and secure.

Control Plane

- Backend services managed by Azure Databricks in **your Databricks account**.
- Hosts the web app UI; handles orchestration, job scheduling, cluster management.
- **Does NOT process your data.**

Compute Plane — two types

	Serverless Compute	Classic Compute
Runs in	Databricks account (not your subscription)	Your Azure subscription (workspace's VNet)
Managed by	Fully managed by Azure Databricks, auto-scales	You manage networking / VNets
Isolation	Network boundaries between workspaces & clusters	Natural isolation via your own subscription
Best for	Simplicity, no infra ops	Full control over networking & security

Note: Classic workspaces are called **“Hybrid workspaces”** in the Azure portal.

3. Workspace Storage

- **Classic workspaces:** have a workspace storage account that lives in your Azure subscription.
- **Serverless workspaces:** use **default storage** — fully managed storage inside the Databricks account, not your subscription.

Two categories of data live in workspace storage (either type):

- **Workspace file system data** — notebooks, SQL queries/dashboards, alerts, repos, libraries, small config files (Python/YAML).
- **Workspace system data** — SQL query results, job run results, notebook revisions, query plans, cluster logs.
- In classic workspaces, both categories sit in the workspace storage account. If UC was auto-enabled, that account also holds the **default workspace catalog** (default schema, usable by all users — governed via Unity Catalog, no direct storage access).
- **DBFS** (Databricks File System, `dbfs:/`) may exist in classic storage — DBFS root & mounts are **legacy/deprecated**. Use Unity Catalog-managed tables and volumes instead.
- Security tip: enable **firewall support** on the workspace storage account to restrict access to approved resources/networks.

4. Unity Catalog Managed Storage

- **Managed storage** = cloud locations where UC stores data/metadata for **managed tables** (structured/tabular) and **managed volumes** (unstructured: images, audio, logs).
- UC owns the **full lifecycle** — location, layout, deletion. Drop a managed table/volume → a **7-day recovery window** (use `UNDROP` for tables), then files are permanently purged within **48 hours**.
- Two goals: physical data isolation (separate containers/paths) and alignment with org structure / compliance needs.

Three levels of managed storage location

Level	Scope	Recommendation
Metastore	Optional fallback for any catalog/schema without its own location	Not auto-created for new UC workspaces; prefer catalog-level
Catalog	Org units, dev/prod stages, data classification	★ Recommended — clear governance boundary
Schema	Individual projects, use cases, team sandboxes	Use when you need finer-grained isolation

Storage location resolution order

Schema location → Catalog location → Metastore location (most specific wins). If none exist, you cannot create managed tables/volumes.

Storage root vs. storage location

- You specify a **storage root** path; UC auto-appends hashed subdirectories so nothing collides:

- Catalogs → `__unitystorage/catalogs/<uuid>` | Schemas → `__unitystorage/schemas/<uuid>`
- Storage root + generated subdirectory = actual **storage location**. Multiple catalogs/schemas can safely share the same root.
- **Overlap prevention:** UC validates a new managed storage path doesn't overlap existing managed locations, external tables, or external volumes — rejects the operation if it does.

5. External Storage

- **External location** = a Unity Catalog object defining a secure connection to cloud storage = **cloud storage path + storage credential**.
- **Storage credential** = the authentication mechanism (Azure managed identity or service principal). Defined once, referenced by multiple external locations sharing the same security boundary.
- Can point to: Azure Data Lake Storage containers (native, read/write), AWS S3 buckets (read-only), Cloudflare R2 buckets.

Why external locations matter

- **External tables/volumes:** register data that already lives in cloud storage, managed outside UC. Files stay put; UC just governs access. Good for large existing datasets or data shared across systems.
- **Managed storage locations:** an external location's path can be assigned as the managed storage location for a catalog/schema — UC then manages the data lifecycle, but it physically resides in a path you specify.

Aspect	External Tables / Volumes	Managed Storage Locations
Data management	You manage the data files	Unity Catalog manages the data
Data location	Stays in its original location	Stored in the location you specify
Use case	Existing or shared data	New data managed by Unity Catalog

Storage credentials

- Metastore-level objects, available to all attached workspaces by default; only privileged users can use them to create external locations.
- **Azure managed identities** = recommended: no passwords/secrets to rotate, works with firewall-protected storage.
- **Service principals** = legacy alternative: requires manual Entra ID app + secret rotation.

Workspace binding

- Restricts an external location (or storage credential) to specific workspaces, regardless of a user's Unity Catalog privileges — an extra access-control layer.
- Common pattern: bind **production** credentials/locations to production workspaces only, keeping dev environments out even for privileged users.

6. Default Storage

- Fully managed object storage **built into Azure Databricks** — no external storage account or credentials to configure.
- Used by both classic and serverless workspaces for internal features: **Data Classification, Anomaly Detection, Clean Rooms, Knowledge Assistant.**
- In **serverless workspaces**, also the primary storage for workspace system data and for catalogs you create. A new serverless workspace auto-provisions a **default catalog** on default storage.

Availability

- Creating **new** catalogs in default storage: serverless workspaces only.
- Classic workspaces can read default-storage catalogs, but only via **serverless compute** — classic compute clusters cannot access them.

Benefits vs. considerations

Area	Benefit	Consideration
Setup	No storage accounts/credentials to configure	Serverless-only; not for orgs needing full infra control
Compute	Instant, scalable serverless compute	Classic compute clusters can't read/write it at all
Governance	Unity Catalog privilege model, SQL GRANT	No cloud-native RBAC integration
Isolation	Catalogs private to the creating workspace by default	Cross-workspace access needs explicit catalog binding
Lifecycle	Auto-cleans managed table/volume files	Only applies to managed objects, not external tables
External access	BI tools (Power BI, Tableau) via ODBC/JDBC	No direct Delta/Iceberg file reads — use external storage for that

Rule of thumb: default storage → new serverless workloads & dev environments. External storage → production needing direct file access or classic compute.

7. Key Terms at a Glance

Term	Definition
Metastore	Account-level, region-specific container for Unity Catalog metadata + permissions
Catalog	Top level of the 3-level namespace; typically an org unit/env/classification
Schema	Mid level of the namespace; a project, use case, or team sandbox
Managed table/volume	UC controls storage location + full lifecycle
External table/volume	Data lives & is managed outside UC; UC only governs access
Storage credential	Auth mechanism (managed identity/service principal) for cloud storage
External location	Storage credential + cloud path, registered as a UC securable object
Default storage	Databricks-managed storage built into the account; serverless-only for new catalogs
DBFS	Legacy Databricks File System (dbfs:/) — deprecated, avoid for new work
Workspace binding	Restricts a credential/location to specific workspaces

---

Sources: Microsoft Learn training module "Understand Azure Databricks architecture," cross-checked against current docs.microsoft.com/azure/databricks pages (High-level architecture, Default storage, Specify a managed storage location, Object storage lifecycle) — verified July 2026.